Scenario 05

🌐 **API Testing**

✗

"Write API test cases for user registration."

What's missing

No HTTP method, no request/response structure, no status codes to validate, no tool context (Postman/RestAssured).

context : Act as an experienced QA Engineer.

Instructions

[MANDATE] Identify all required and optional input fields for user registration.

[MANDATE] Mention preconditions, business rules, validations, and dependencies.

[MANDATE] Clearly define the expected API response, status code, response body, and database/user account behavior.

[MANDATE] Positive Scenarios: Write valid registration test cases.

[MANDATE] Negative Scenarios: Write invalid, boundary, duplicate, missing field, and security-related test cases.

[MANDATE] Test Data: Provide realistic test data for each test case.

API Details:

Endpoint: POST /api/v1/register

Request body:

{

"firstName": "Aish",

"lastName": "S",

"email": "aishwarya.test@gmail.com",

"password": "StrongPass@123",

"confirmPassword": "StrongPass@123",

"phoneNumber": "+919876543210",

"dateOfBirth": "1990-08-27",

"country": "India"

}

Example:

Test Case ID

Scenario

Category

Preconditions

Input/Test Data

Steps

Expected Status Code

Expected Result

Priority

Include functional, validation, boundary, security, duplicate user, password policy, email format, phone number, DOB age restriction, mandatory fields, and API response validation test cases.

and API response validation test cases.

Persona

Act as a Test lead to review the testcases

Output

Downloadable excel

Tone:

Professional Tone with clarity

Inference

Difference: more detailed a clear test case. Clear steps and a proper rendition of process

More test cases lesser assumptions

More status code validations

Better clarity on the data and expected results

Scenario 04

2. "Write test cases for user roles."

"Write test cases for user roles."

What's missing

No role types (Admin, Manager, Viewer), no permission matrix, no unauthorized access scenarios, no app domain.

ICEPOT Prompt

I – Instructions
Write detailed test cases for **User Roles and Permissions**.

Cover:

- Role creation
- Role assignment
- Role update
- Role deletion

- Permission validation
- Unauthorized access
- Role-based UI visibility
- API-level access control
- Permission matrix validation

C – Context

The application is a **Project Management Web Application** where users can manage projects, tasks, reports, and team members.

Assume the following role types:

Role	Description
Admin	Full access to all modules and user management
Manager	Can manage projects, tasks, and team members but cannot manage system settings
Viewer	Read-only access to assigned projects and reports

Permission Matrix:

Feature	Admin	Manager	Viewer
Create Project	Yes	Yes	No
Edit Project	Yes	Yes	No
Delete Project	Yes	No	No
View Project	Yes	Yes	Yes
Create Task	Yes	Yes	No
Edit Task	Yes	Yes	No
Delete Task	Yes	Yes	No
View Reports	Yes	Yes	Yes
Manage Users	Yes	No	No
Manage Roles	Yes	No	No

Feature	Admin	Manager	Viewer
System Settings	Yes	No	No

Include positive, negative, boundary, security, and unauthorized access scenarios.

E – Example
Example Scenario:

A user with **Viewer** role tries to delete a project.

Expected Result:
The system should deny the action, show an authorization error, and no project should be deleted.

P – Persona
Act as:

1. Senior QA Engineer
2. Security QA Tester
3. Business Analyst
4. API Tester

Think from functional, UI, API, security, and business-rule perspectives.

O – Output
Provide output in a table format with the following columns:

| Test Case ID | Role | Feature/Permission | Scenario | Preconditions | Steps | Expected Result | Access Control Validation |

T – Tone
Use a professional QA tone. Keep the test cases detailed, practical, and interview-ready.

INFERENCE:

Result is more theoretical. No cases generated but idea to create cases is generated. Once the detailed prompt given the results has more combination of cases

Scenario 06

"Test OTP login on mobile."

ICEPOT Prompt

I – Instructions

Create detailed mobile test cases for **OTP Login on Mobile**.

Cover:

* OTP generation
* OTP verification
* OTP expiry
* Resend OTP behavior
* Invalid OTP
* Expired OTP
* Network failure
* App background/foreground behavior
* Device permission behavior
* Android and iOS platform coverage
* Appium automation considerations

C – Context

The feature is an OTP-based login flow in a mobile application. The user enters a mobile number, receives an OTP, enters the OTP, and logs in successfully if the OTP is valid.

Assume:

* Platforms: Android and iOS
* Automation Tool: Appium

* Devices: Android real device, Android emulator, iPhone real device, iOS simulator

* OTP Length: 6 digits

* OTP Expiry Time: 5 minutes

* Resend OTP available after 30 seconds

* Maximum OTP Attempts: 3

* Network types: Wi-Fi, mobile data, poor network, offline mode

* Android supports SMS auto-read

* iOS requires manual OTP entry or keyboard suggestion

E – Example

Example scenario:

User enters mobile number `+919876543210`, receives OTP `123456`, enters the OTP within 5 minutes, and taps **Login/Verify**.

Expected result:

User should be logged in successfully and redirected to the Home/Dashboard screen.

P – Persona

Act as:

1. A Senior Mobile QA Engineer

2. An Appium Automation Tester

3. A Security QA Tester

4. A Product QA Analyst

Think from functional, negative, boundary, security, device compatibility, and automation perspectives.

O – Output

Provide the output in a table format with the following columns:

| Test Case ID | Platform | Scenario | Test Data | Preconditions | Steps | Expected Result | Appium Automation Notes |

Include positive, negative, boundary, security, resend OTP, expiry, network failure, and device compatibility scenarios.

T – Tone

Use a professional QA tone. Keep the test cases clear, detailed, practical, and interview-ready.

INFERENCE: More test cases